

DFS + Backtracking Pattern Sheet (Grid Problems)

1. Rat in a Maze (All Paths)

Problem

Given a maze of 1 (open) and 0 (blocked),

Find **all possible paths** from $(0,0)$ to $(n-1,n-1)$.

Moves:

- D
- L
- R
- U

Cannot visit a cell twice.

Core Idea

Need to explore **every possible path**.

Whenever destination is reached,

store the current path.

DFS Meaning

```
dfs(r, c, path)
```

State contains

- Current position
 - Path string built so far
-

Base Cases

```
Invalid
```

```
↓
```

```
return
```

Destination

↓

```
res.append(path)
```

Backtracking

```
vis[r][c] = True
```

Explore

```
vis[r][c] = False
```

Return Type

Nothing

Answer stored in

```
res.append(path)
```

2. Word Search (LC 79)

Problem

Given a board and a word,

Return **True** if the word exists.

Cannot reuse a cell.

New Idea

Instead of carrying

```
Path String
```

carry

```
Word Index
```

DFS Meaning

```
dfs(r, c, index)
```

Meaning

```
Can I match the remaining word starting from this cell?
```

Base Cases

Invalid

```
return False
```

Matched entire word

```
return True
```

Backtracking

```
vis[r][c]=True
```

Explore

```
vis[r][c]=False
```

Combine

```
return (  
    dfs(...)  
    or  
    dfs(...)  
    or  
    dfs(...)  
    or  
    dfs(...)  
)
```

Need only one successful path.

3. Path with Maximum Gold (LC 1219)

Problem

Each cell contains gold.

Collect maximum gold.

Cannot revisit cells.

Can start from **any gold cell**.

New Idea

Unlike previous problems,

there is **no fixed starting point**.

Run DFS from every gold cell.

DFS Meaning

```
dfs(r,c)
```

Meaning

Maximum gold collectable starting from this cell.

Base Case

Invalid

```
return 0
```

Transition

```
best=max(  
    up,  
    down,  
    left,  
    right  
)
```

Return

```
grid[r][c]+best
```

Current gold

-

Best future path.

Outer Loop

```
for every cell:  
    if gold:  
        ans=max(ans,dfs(cell))
```

4. Unique Paths III (LC 980)

Problem

Grid contains

1 → Start
2 → End
0 → Empty
-1 → Obstacle

Visit **every non-obstacle cell exactly once**.

Return number of valid paths.

New Idea

Reaching destination



is NOT enough.

Need to visit

every walkable cell exactly once.

Logic

Count

Total Walkable Cells

Store as

remaining

Find

Start Cell

Run DFS.

Every visit

```
remaining-=1
```

When reaching destination

Valid only if

```
remaining==1
```

DFS Meaning

```
dfs(r,c,remaining)
```

Meaning

Number of valid paths from current cell while covering remaining cells.

Base Cases

Invalid

```
return 0
```

Destination

```
return 1 if remaining==1 else 0
```

Combine

```
return  
dfs(up)  
  
+  
dfs(down)  
  
+
```

```
dfs(left)

+

dfs(right)
```

Need total number of paths.

Pattern Comparison

Problem	DFS State	Goal	Return	Combine
Rat in Maze	<code>(r,c,path)</code>	Reach destination	None	<code>append(path)</code>
Word Search	<code>(r,c,index)</code>	Match entire word	<code>bool</code>	<code>or</code>
Maximum Gold	<code>(r,c)</code>	Maximum gold	<code>int</code>	<code>max()</code>
Unique Paths III	<code>(r,c,remaining)</code>	Visit every cell	<code>int</code>	<code>+</code>

Common DFS + Backtracking Template

```
def dfs(state):  
  
    # Invalid  
    if invalid:  
        return base_value  
  
    # Goal  
    if goal:  
        return/update answer  
  
    # Choose  
    vis[current]=True  
  
    # Explore  
    ans = combine(  
        dfs(...),  
        dfs(...),  
        dfs(...),  
        dfs(...)  
    )  
  
    # Undo  
    vis[current]=False  
  
    return ans
```

How to Identify the Pattern

If the problem says

- ✓ Grid
- ✓ Cannot revisit cells
- ✓ Explore all possibilities

↓

Think

DFS + Backtracking

Then ask

What should DFS return?

If question asks

Question	Combine
Does a path exist?	<code>or</code>
Longest path?	<code>max()</code>
Shortest path?	<code>min()</code>
Count all paths?	<code>+</code>
Store every path?	<code>append()</code>

Recognition Checklist

Grid / Graph

↓

Visit once?

↓

Yes

↓

Need every possibility?

↓

Yes

↓

DFS

+

Visited Array

+

Backtracking

↓

Ask:

"What should `dfs(state)` return?"

↓

Choose combine operation

or

max()

min()

+

append()

Key Lesson

The **DFS skeleton never changes**.

Only these four things change:

1. **State** carried by DFS.
2. **Base case**.
3. **What DFS returns**.
4. **How child answers are combined**.

Master these four, and most grid backtracking interview problems become variations of the same pattern.